ЛАБОРАТОРНАЯ РАБОТА №6: ОБМЕН СООБЩЕНИЯМИ В MPI

Выполнил: Заика Тимур, группа 11-404

1. Логика обмена и теги

В основе программы лежит схема «звезда»: нулевой процесс (корень) поочередно отправляет данные каждому из остальных процессов (ранги от 1 до *P-1*), а затем ожидает от них ответных сообщений. В минимальной конфигурации из двух процессов это просто двусторонний пинг-понг. Для корректной фильтрации сообщений используются теги: `TAG_MESSAGE` (равен 1) маркирует данные от корня, а `TAG_REPLY` (равен 2) — ответы от рабочих процессов. Теговый механизм гарантирует, что процесс не перепутает входящий запрос с ответом. В режиме бенчмарка (замера времени) оба процесса обмениваются массивами символов с теми же тегами.

Шаг	Ранг	Выполняемая операция	Партнер	Используемый тег
1	0	MPI_Send: отправка строки (MPI_CHAR)	Ранг r	TAG_MESSAGE = 1
2	r	MPI_Recv: получение и вывод строки	Ранг 0	TAG_MESSAGE = 1
3	r	MPI_Send: отправка ответа «Message received by rank r»	Ранг 0	TAG_REPLY = 2
4	0	MPI_Recv: получение и вывод ответа	Ранг r	TAG_REPLY = 2

2. Сборка и запуск

Среда выполнения: MS-MPI в окружении MSYS2, компилятор GCC 16.2.0. Тестирование проводилось на процессоре Intel Core i5-12450H.

```
mpicc point_to_point.c -o point_to_point.exe
mpiexec -n 2 ./point_to_point.exe # классический обмен между ранками 0 и 1
mpiexec -n 4 ./point_to_point.exe # корень общается со всеми
mpiexec -n 2 ./point_to_point.exe time # тестирование задержек
```

4. Заключение

В MPI каждый процесс идентифицируется уникальным целым числом — рангом (от 0 до P-1 в рамках коммуникатора MPI_COMM_WORLD). При вызове MPI_Send мы явно указываем, кому шлем данные, а в MPI_Recv — от кого ожидаем. Поскольку код программы един для всех, именно ранг определяет ветку исполнения (логику «если я корень — шлю, иначе — жду»). Использование тегов — это надежный способ разделения потоков данных: вызов MPI_Recv с тегом TAG_REPLY не перехватит первоначальное сообщение, так как у них разные идентификаторы. Процесс 0 опрашивает получателей последовательно (по возрастанию ранга), поэтому даже если рабочие процессы отправят ответы в хаотичном порядке, корень обработает их строго по порядку. Анализ замеров (Таблица 3) показывает, что передача малых объемов данных (до 1 КБ) ограничивается задержками сети/шины и занимает около 1 мкс, тогда как передача массива в 1 МБ требует уже 0.22 мс.

3. Результаты работы

Таблица 1. Тестирование базового обмена (2 процесса)

Номер попытки	Кол-во процессов	Данные доставлены?	Ответ получен?	Завершение без ошибок?
1	2	Да	Да	Да
2	2	Да	Да	Да
3	2	Да	Да	Да

Если запустить программу на одном процессе, срабатывает защита: выводится предупреждение о необходимости как минимум двух процессов, и программа корректно завершается.

Таблица 2. Масштабирование обмена (процесс 0 общается со всеми)

Всего процессов (P)	Исходящих сообщений	Успешных ответов	Обмен завершен полностью?
2	1	1	Да
4	3	3	Да
8	7	7	Да

Таблица 3. Замер времени Round-Trip Time (RTT) между процессами 0 и 1

Перед началом замеров выполняется 10 холостых обменов. Время измеряется с помощью `MPI_Wtime`.

Размер пакета (байт)	Количество итераций	Среднее время обмена (сек)
1	1000	$7.94 \cdot 10^{-5}$
1024	1000	$1.24 \cdot 10^{-5}$
65536	1000	$3.67 \cdot 10^{-5}$
1048576	1000	$2.22 \cdot 10^{-4}$